

Sistema de Monitoreo de Temperatura y Humedad con alerta sonora

Paola Andrea Chalarca Restrepoy Kevin Restrepo Orrego
Electrónica digital II

Índice

1. Identificación del proyecto	2
2. Resumen	2
3. Descripción del hardware	2
4. Descripción del software	3
5. Estructura del código	3
6. Explicación de funciones principales	4
7. Comunicación (si aplica)	4
8. Interfaz de usuario	4
9. Procedimiento de prueba	5
10. Manejo de errores y seguridad	5

1. Identificación del proyecto

- **Nombre del proyecto:** Sistema de monitoreo de temperatura, humedad y movimiento con alerta integrada
- **Integrantes del equipo:** Paola Chalarca y Kevin Restrepo

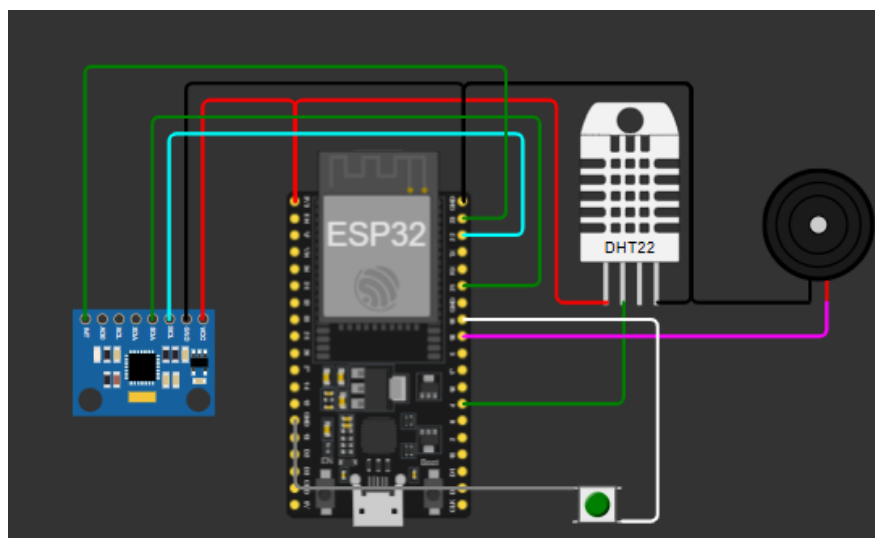
2. Resumen

Este proyecto consiste en un sistema con ESP32 que mide temperatura y humedad con un sensor DHT11 y detecta movimiento con un MPU6050. El sistema toma estos datos y activa un buzzer cuando hay alguna condición fuera de lo normal. Además, se conecta a WiFi para enviar información a un servidor y permite consultar los datos mediante un bot de Telegram. También cuenta con un botón de pánico que genera una alerta inmediata.

3. Descripción del hardware

- ESP32 como unidad de procesamiento principal.
- Sensor DHT11 para medición de temperatura y humedad.
- Sensor MPU6050 para detección de movimiento mediante acelerómetro.
- Buzzer para alertas sonoras.
- Botón de pánico conectado a una interrupción

Diagrama de conexión:



4. Descripción del software

El software está desarrollado en MicroPython y ejecutado sobre el ESP32. Se encarga de:

- Leer los sensores periódicamente.
- Evaluar condiciones de alerta según estándares definidos.
- Hacer interrupciones (botón de pánico y movimiento).
- Enviar datos a un servidor mediante HTTP.
- Permitir interacción remota mediante un bot de Telegram.

5. Estructura del código

- Archivo principal: `main.py`.
- Se definen primero las configuraciones generales: WiFi, Telegram y servidor.
- Selección de pines, sensores (DHT11 y MPU6050) y variables globales.
- Configuración de interrupciones para el botón de pánico y el sensor de movimiento.
- Definición de funciones para:
 - Conexión a WiFi.
 - Lectura de sensores.
 - Detección de movimiento.
 - Alertas.
 - Envío de datos al servidor.
 - Manejo de mensajes de Telegram.
- Bloque de inicialización donde se conecta el sistema y se configuran los sensores.
- Bucle principal `while True` que ejecuta continuamente:
 - Verificación de interrupciones.
 - Lectura periódica de sensores.
 - Envío de datos.
 - Atención de comandos desde Telegram.

6. Explicación de funciones principales

- `conectar_wifi()`: Establece la conexión a la red inalámbrica.
- `configurar_mpu()`: Configura el sensor MPU6050 para generar interrupciones.
- `leer_sensores()`: Obtiene datos de temperatura, humedad y movimiento.
- `detectar_movimiento()`: Calcula la magnitud de aceleración para identificar movimiento.
- `verificar_alertas()`: Evalúa condiciones críticas y activa el buzzer.
- `enviar_datos()`: Envía información al servidor local.
- `verificar_boton_panico()`: Gestiona la activación del botón de pánico.
- `mensajes()`: Procesa comandos recibidos desde Telegram.

7. Comunicación (si aplica)

El sistema utiliza diferentes métodos de comunicación:

- WiFi para conexión a red local.
- Protocolo HTTP para envío de datos al servidor.
- API de Telegram para comunicación remota con el usuario.

8. Interfaz de usuario

- Interfaz física:
 - Buzzer para indicar alertas.
 - Botón de pánico como entrada directa.
- Interfaz remota:
 - Bot de Telegram que permite consultar información mediante comandos.
- Interfaz de monitoreo:
 - Consola serial para visualización de datos.

9. Procedimiento de prueba

1. Encender el ESP32 y verificar conexión WiFi.
2. Comprobar lectura del sensor DHT11.
3. Generar movimiento para validar el MPU6050.
4. Activar el botón de pánico y verificar respuesta.
5. Confirmar activación del buzzer en condiciones de alerta.
6. Enviar comandos desde Telegram y validar respuestas.
7. Verificar envío correcto de datos al servidor.

10. Manejo de errores y seguridad

- Uso de estructuras `try-except` para evitar fallos en comunicación.
- Control de intentos en conexión WiFi.
- Uso de interrupciones para eventos críticos.
- Actualización periódica de datos para evitar inconsistencias.
-
- No se implementan medidas de seguridad avanzadas en la comunicación, ni mecanismos automáticos para reiniciar el sistema en caso de fallos.